



OpenViking 通用知识库

技术说明文档

面向 AI 应用的文档处理、记忆处理与检索增强底座

项目	说明
文档类型	技术架构说明 / 实施设计说明
项目定位	企业通用知识库与上下文数据库，支撑客服机器人、智能投标等 AI 应用
版本口径	基于当前工作区源码与 docs/zh/concepts 文档梳理，生成日期 2026-04-27
核心流程	文档处理、语义摘要与向量化、会话记忆沉淀、意图检索、分层召回与重排
主要读者	评审专家、研发负责人、AI 应用集成方、运维与交付团队

核心判断

OpenViking 的价值不止于当前客服机器人，而在于把文档、经验、资质、历史对话和工具使用痕迹统一沉淀为可检索、可复用、可治理的知识与记忆底座。智能客服是成功案例，智能投标、售前方案、交付知识助手等场景可在同一底座上扩展。

1. 文档概览

本文说明 OpenViking 通用知识库的技术架构、关键数据结构和端到端处理流程。文档重点围绕三个问题展开：第一，外部资料如何被解析为可检索的知识树；第二，对话过程中的经验如何沉淀为长期记忆；第三，AI 应用如何基于会话意图、权限边界和多层索引完成精准检索。

设计目标

OpenViking 不是单一问答机器人，而是 AI 应用的知识与上下文基础设施：上层应用可以是客服、投标、售前、交付助手或企业内部 Copilot；底层统一处理知识入库、记忆沉淀、语义索引、检索召回和多租户隔离。

能力域	解决的问题	对应模块
文档处理	把 PDF、Word、网页、代码仓库、媒体等资料转换为结构化知识树	parse、TreeBuilder、ResourceService
语义加工	为目录和文件生成 L0/L1 摘要，并写入向量索引	SemanticQueue、SemanticProcessor、EmbeddingQueue
记忆处理	把会话、工具调用、业务经验沉淀为用户/Agent 长期记忆	Session、SessionCompressor、MemoryExtractor、MemoryDeduplicator
检索增强	依据用户问题和会话上下文，在资源、记忆、技能中分层召回	SearchService、IntentAnalyzer、HierarchicalRetriever、Rerank
治理与隔离	保证账号、用户、Agent、会话边界清晰，支持异步任务与状态跟踪	RequestContext、Auth、TaskTracker、KnowledgeDocumentRegistry

1.1 总体架构

```
Client / AI App
| Local Client, HTTP API, Console, MCP, Bot
v
Service Layer
|-- ResourceService: add_resource, add_skill, wait_processed
|-- SearchService: find, search, grep, glob
|-- SessionService: message, used context, commit, archive
|-- FS/Content/Relation/Pack/Debug services
v
Core Pipelines
|-- Parser -> TreeBuilder -> SemanticQueue -> Vector Index
|-- Session -> Compressor -> MemoryExtractor -> Dedup -> Memory Store
|-- IntentAnalyzer -> HierarchicalRetriever -> Rerank -> MatchedContext
v
Storage
|-- VikingFS / AGFS: L2 原文、目录、.abstract.md、.overview.md
|-- VectorDB: L0/L1/L2 语义索引、score、metadata
|-- Task/Registry: 入库状态、异步任务、处理遥测
```

架构采用“内容存储 + 向量索引”的双层设计。VikingFS/AGFS 保存原始内容、目录树、摘要文件和元数据；向量库保存可检索记录、层级信息和权限字段。这样既保留可审计的知识来源，又能为 AI 应用提供低延迟语义召回。

1.2 核心概念

概念	说明
Viking URI	统一资源标识，例如 viking://resources、viking://user/memories、viking://agent/skills、viking://session/{user}/{session_id}。
ContextType	resource、memory、skill 三类上下文，分别对应企业资料、长期记忆和可执行/可引用技能。
ContextLevel	L0 abstract、L1 overview、L2 detail/content。L0 用于快速筛选，L1 用于目录导航与重排，L2 用于最终引用和回答。
RequestContext	携带 account/user/agent/role，用于服务鉴权、目录初始化、向量过滤和多租户隔离。
SemanticMsg	语义加工队列消息，描述待处理 URI、上下文类型、目标 URI、处理状态、锁和遥测信息。
MatchedContext	检索返回的知识命中结果，包含 URI、层级、分数、来源和可选 provenance。

2. 文档处理流程

文档处理流程负责把外部知识源转化为 OpenViking 内部可检索、可治理的知识树。该流程采用同步接收、异步语义加工的架构：用户上传或指定远程资源后，系统先完成格式解析与临时目录构建，再通过队列完成摘要、向量化和索引更新。

```
用户/系统提交资源
-> /api/v1/resources 或 Local Client add_resource()
-> ResourceService 注册知识文档和任务状态
-> Parser Registry 选择解析器
-> Parser 在临时 VikingFS 中生成目录树和 L2 内容
-> TreeBuilder 确定最终 Viking URI
-> SemanticQueue 投递 SemanticMsg
-> SemanticProcessor 自底向上生成 L0/L1
-> VectorDB 写入语义索引
-> KnowledgeDocumentRegistry 更新状态
```

2.1 资源接入

- HTTP 接入：/api/v1/resources 支持 path、temp_file_id、to、parent、folder_path、instruction、wait、timeout、include/exclude、watch_interval 等参数。
- 临时上传：/api/v1/resources/temp_upload 先写入上传目录，再由 temp_file_id 转为可处理的本地临时文件。
- 本地接入：Python Local Client 和 Sync/Async HTTP Client 对上层业务隐藏路由细节，常用方法包括 add_resource、add_skill、wait_processed、find、search、read、abstract、overview。
- 远程资源保护：HTTP 模式默认要求远程资源来源，只有 temp_file_id 场景允许解析上传后的本地临时路径，降低服务端任意文件读取风险。

输入类型	解析结果	说明
Markdown / TXT / HTML	保持标题层级、正文块、链接和基础结构	适合作为知识库基础文档
PDF / Word / Excel / PowerPoint	转换为可读文本、表格或页面结构	适合制度、方案、投标资料、产品手册
代码仓库 / 目录 / ZIP	保留目录结构，按文件类型识别代码与文档	支持源码知识库、SDK 文档、工程经验沉淀
图片 / 音频 / 视频	生成多模态摘要并映射到资源目录	用于截图、录屏、会议资料等非纯文本信息
飞书 / EPUB / 旧版文档	通过专用解析器或转换器进入统一树结构	便于企业已有资料迁移

2.2 Parser 与切分策略

Parser 的职责是格式转换和结构化切分，不承担 LLM 摘要生成。这个边界很重要：解析阶段尽可能确定、可重试、低成本；语义阶段再异步调用模型，避免大文件入库阻塞和内存压力。

1. Parser Registry 根据文件扩展名、资源类型和参数选择解析器，例如 markdown.py、pdf.py、word.py、excel.py、powerpoint.py、directory.py、zip_parser.py、repository 相关解析逻辑等。
2. 解析器把原始资料写入临时 VikingFS，形成一个文档根目录，目录内包含可读 L2 内容文件和必要的结构文件。
3. 长文档按标题、章节、token 预算和文件类型切分；小片段会合并，过大片段会继续拆分，目标是在保持语义完整性的同时控制后续摘要和向量化成本。

4. 代码类资料会识别语言和文件类型，必要时结合 AST、LLM 或 AST+LLM 模式生成更有结构的代码摘要。

2.3 TreeBuilder 与 URI 落位

TreeBuilder 读取临时目录中的唯一文档根节点，清洗文件名，确定最终 URI。如果用户指定 to，则写入精确目标；如果指定 parent，则在父目录下创建资源；否则根据 scope 自动落到 viking://resources、viking://user 或 viking://agent。媒体资源会根据类型进入对应资源空间。若目标名称冲突，系统会自动追加 _1、_2 等后缀生成唯一 URI。

工程价值

临时目录 + 最终 URI 的两阶段设计，使解析失败不会污染正式知识库；同时也为增量同步、生命周期锁、watch 任务和处理状态回滚保留了清晰边界。

2.4 语义加工与向量化

资源树落位后，SemanticQueue 投递 SemanticMsg。SemanticProcessor 以目录为单位自底向上工作：先为叶子文件生成摘要，再汇总子目录摘要，生成当前目录的 .overview.md 和 .abstract.md，最后写入向量索引。系统支持增量更新：当 target_uri 已存在且与当前临时 URI 不同，会进入增量同步模式，只处理新增、修改和删除部分。

层级	内容	检索作用
L0 .abstract.md	高度压缩的节点摘要，约百 token 量级	用于快速向量召回、候选粗筛、低成本相关性判断
L1 .overview.md	目录级概览，汇总子节点摘要和结构信息	用于层级导航、重排、回答前定位上下文
L2 原始内容	文档正文、代码、表格转换文本、媒体摘要或原文件引用	用于最终引用、证据展开和生成答案

- 并发控制：SemanticProcessor 通过 max_concurrent_llm 控制摘要生成并发，避免模型服务过载。
- 熔断与重试：模型 API 出现临时错误时会重新入队，永久错误会记录失败并更新文档处理状态。
- 生命周期锁：资源同步与语义处理可持有 subtree lock，降低并发写入导致的不一致风险。
- 状态可观测：ResourceService、KnowledgeDocumentRegistry、QueueManager 和 telemetry 共同记录入库、等待、失败、队列耗时等指标。

3. 记忆处理流程

记忆处理将一次次 AI 交互转化为可长期复用的用户画像、偏好、业务实体、事件、Agent 案例、工具模式和技能经验。它解决的问题是：AI 应用不能只依赖静态资料，还需要从真实服务和业务执行中持续学习。

Session 创建和交互

- > add_message 写入用户/助手/工具消息
- > used 记录本轮引用过的上下文或技能
- > commit_async 立即归档当前消息
- > 后台任务生成 archive L0/L1 摘要
- > MemoryExtractor 抽取长期记忆候选
- > 向量预筛 + MemoryDeduplicator 去重/合并
- > 写入 viking://user/memories 或 viking://agent/memories
- > 更新 active_count 与 .done/.failed 状态

3.1 Session 生命周期

阶段	关键动作	技术要点
创建	SessionService 创建 session 目录和 .meta.json	URI 形如 viking://session/{user_space}/{session_id}
交互	add_message 追加 TextPart、ContextPart、ToolPart 等消息片段	消息保存为 JSONL，便于增量读取与审计
引用记录	used 记录本轮使用的资源、记忆或技能 URI	commit 后可更新 active_count，支持热度治理
提交	commit_async 将当前消息移入 history/archive_xxx	同步阶段快速返回 task_id，异步阶段处理摘要和记忆
归档完成	写入 .abstract.md、.overview.md、.done	失败时写 .failed.json，并阻断后续依赖归档以避免链路错乱

3.2 两阶段提交机制

Session commit 分为同步和异步两阶段。同步阶段不调用 LLM：它只递增 compression_index，将消息写入 archive_xxx/messages.jsonl，清空当前消息，并立即返回 task_id。异步阶段再生成 archive 摘要、抽取长期记忆、更新活跃度和写完成标记。这种设计让对话主链路不被模型摘要耗时拖慢。

提交阶段	输入	输出	失败处理
Phase 1 同步归档	当前未提交消息、使用记录快照	archive URI、task_id、当前消息清空	若无消息则返回 archived=false
Phase 2 异步压缩	archive messages、上一归档 overview	.abstract.md、.overview.md、.meta.json	写 .failed.json，后续归档可检测阻断
长期记忆抽取	归档消息、工具调用、上下文引用	CandidateMemory 列表	抽取失败不影响已归档原始记录
去重与合并	候选记忆 + 相似历史记录	create / merge / skip / delete 决策	保留可审计的候选与处理日志

3.3 记忆类别与作用

类别	归属	示例	作用
profile	用户	岗位、部门、职责、常用系统	形成稳定用户画像
preferences	用户	回答风格、格式偏好、常用语言	提升个性化交互体验
entities	用户	客户、项目、产品、合同、酒店集团	让后续检索可关联业务实体
events	用户	会议结论、重要交付节点、历史问题	保留业务时间线
cases	Agent	客服问题闭环、投标案例、方案模板	复用成功实践
patterns	Agent	常见处理策略、排障路径	沉淀可复用工作方法
tools	Agent	工具调用参数、成功率、耗时、token	优化工具选择和调用策略
skills	Agent	可迁移技能说明、触发条件	把经验升级为可复用能力

3.4 去重、合并与热度治理

- MemoryExtractor 先把归档消息转为结构化候选，工具调用会被格式化为 JSON 片段，便于模型理解输入、输出、耗时和调用结果。
- MemoryDeduplicator 会基于类别决定目标空间：用户类记忆进入 user memories，Agent 类记忆进入 agent memories。
- 系统先通过向量相似度预筛历史记忆，再交由 LLM 判断是否创建新记忆、合并旧记忆、跳过候选或删除冗余记忆。
- 长记忆会按 memory_chunk_chars 和 overlap 切块，既保留完整 L2 记录，也让局部内容能被检索命中。
- active_count 会随着会话引用更新，后续可用于记忆归档、冷热分层和低价值记忆治理。

对 AI 应用的意义

客服机器人通过记忆沉淀常见问题、客户环境和处理经验；智能投标则可以沉淀历史投标策略、资质使用经验、行业客户关注点和方案复用模式。二者共享同一套记忆管线。

4. 检索流程

OpenViking 的检索并非简单向量搜索，而是结合会话上下文、检索意图、知识类型、层级结构和重排策略的分层召回流程。上层应用可按场景选择 `find` 或 `search`：`find` 适合低延迟单次查询，`search` 适合结合会话语义的复杂任务。

接口	是否使用会话	是否做意图分析	适用场景
<code>find(query)</code>	否	否	直接在指定 <code>target_uri</code> 或全局空间中做语义查找，适合搜索框、引用定位、快速问答
<code>search(query, session_id)</code>	可选	是	读取会话摘要和最近消息，生成多条 <code>TypedQuery</code> ，适合多轮对话和复杂业务任务
<code>grep(pattern)</code>	否	否	基于文本模式查找原文，适合精确关键字、编号、配置项
<code>glob(pattern)</code>	否	否	基于路径模式查找文件或目录，适合结构浏览和批量处理

4.1 search 的意图分析

`SearchService` 在存在 `session` 时会调用 `session.get_context_for_search(query)`，将压缩后的会话摘要、最近消息和当前问题交给 `IntentAnalyzer`。`IntentAnalyzer` 输出 0 到 5 条 `TypedQuery`，每条包含 `query`、`context_type`、`intent` 和 `priority`。这样一个用户问题可以被拆成资源查询、记忆查询和技能查询。

用户问题：“帮我准备某酒店集团的智能投标方案”

`IntentAnalyzer` 可能拆解为：

1. RESOURCE: 查找产品能力、技术方案、资质材料
2. MEMORY: 查找该客户或同类客户历史偏好
3. MEMORY: 查找历史投标成功案例与失败风险
4. SKILL: 查找可用的标书生成或合规检查技能

4.2 分层召回

`TypedQuery`

- > 根据 `context_type` 选择根目录
 - RESOURCE: `viking://resources`
 - MEMORY: `viking://user/memories`, `viking://agent/memories`
 - SKILL: `viking://agent/skills`
- > 全局向量召回 L0/L1 候选
- > 合并起点并按 `score / priority` 排序
- > 进入目录层级，读取 `abstract / overview`
- > 递归扩展相关子节点和 `related_uri`
- > Rerank 候选
- > 返回 `MatchedContext`

层级检索的优势是既能快速发现相关知识区域，又能沿目录结构向下展开到更精确的原文片段。对大型资料库而言，直接把所有 L2 内容做一次平铺召回容易丢失结构；OpenViking 用 L0/L1 先定位，再进入 L2 展开，能兼顾速度、可解释性和召回质量。

4.3 重排与回退

- 若配置了 rerank provider，系统会对候选文本执行批量重排，提升与当前问题的语义贴合度。
- 若重排服务异常或未配置，检索会回退到向量分数，保证服务可用性。
- score_threshold 和 limit/node_limit 可控制召回质量和数量；include_provenance 可在 API 返回中携带来源线索。
- related_uri 支持知识节点间的显式关联，便于从命中文档跳转到案例、资质、流程或相关技能。

4.4 检索结果在应用中的使用

应用	典型检索动作	生成侧使用方式
客服机器人	从产品手册、FAQ、历史工单和客户记忆中召回答案依据	把命中上下文作为 RAG 引用，减少幻觉并给出可追溯回答
智能投标	召回招标要求、资质材料、成功案例、行业方案和风险条款	生成章节草稿、检查漏项、匹配资质和历史素材
售前方案助手	召回产品能力、行业客户案例和交付边界	组合方案结构，输出面向客户的解决方案
交付知识助手	召回实施手册、故障排查、项目复盘和工具经验	辅助现场人员定位问题并复用历史解决方案

5. 存储、权限与多租户

OpenViking 在存储层把内容、索引、身份边界和任务状态拆开管理。内容侧使用 VikingFS/AGFS 保存可读文件树；索引侧使用向量库保存摘要、分数和 metadata；服务侧通过 RequestContext 将账号、用户、Agent 和角色传递到每次读写与检索操作中。

空间	URI 示例	可见性与用途
资源空间	viking://resources/...	账号内共享知识资料，适合产品手册、制度、方案、资质库
用户记忆	viking://user/memories/...	按用户隔离的画像、偏好、实体和事件
Agent 记忆	viking://agent/memories/...	按 Agent 或 user+agent 范围隔离的案例、模式、工具和技能经验
技能空间	viking://agent/skills/...	Agent 可引用或执行的能力描述
会话空间	viking://session/{user}/{session_id}/...	单会话消息、归档、摘要和任务状态

- HTTP Client 请求头包含 X-API-Key、X-OpenViking-Account、X-OpenViking-User、X-OpenViking-Agent，用于服务端身份解析。
- resources 默认 owner_space 为空，表示账号共享；user、agent、session URI 会根据 UserIdentifier 派生 owner_space。
- 正式多租户模式可通过 root_api_key 和身份头启用；默认模式也保留 default account/user/agent 以便本地和嵌入式部署。
- 向量索引写入 account_id、owner_space、context_type、category、level 等字段，检索时以身份和空间过滤，避免跨用户或跨 Agent 泄漏。

6. API 与集成方式

集成形态	适用情况	说明
Embedded Python	AI 应用与 OpenViking 同进程部署	适合本地工具、实验环境、低延迟私有服务
HTTP Server	多个应用共享知识库服务	通过 FastAPI 路由提供资源、搜索、会话、文件系统等接口
Console / Admin UI	知识管理和运维操作	用于上传资料、查看任务、配置和调试
CLI / SDK	脚本化导入、测试、批处理	适合资料迁移、构建索引、自动化验证
Bot / MCP	连接具体 AI 终端或 Agent 框架	让客服机器人、投标助手等上层应用复用同一知识底座

能力	HTTP 路由/方法	主要用途
资源入库	POST /api/v1/resources	上传或引用资源，触发解析、摘要和索引
临时上传	POST /api/v1/resources/temp_upload	上传本地文件，返回 temp_file_id
语义检索	POST /api/v1/search/find	无会话语义搜索
会话检索	POST /api/v1/search/search	结合 session 的多意图检索

文本/路径检索	POST /api/v1/search/grep, /glob	精确文本查找和路径匹配
会话提交	SessionService commit_async	归档消息并触发长期记忆抽取
内容读取	read / abstract / overview	读取原文、L0 摘要或 L1 概览

7. 运行、可观测性与风险控制

OpenViking 的关键处理链路大量使用异步队列和后台任务，因此运行质量取决于模型服务、向量库、文件系统、队列状态和任务注册表的协同。技术说明和实施交付中应重点关注以下运维点。

关注项	指标/现象	处理建议
资源入库耗时	wait_complete 超时、 queue.wait.duration_ms 偏高	检查 Parser 耗时、LLM 并发、向量库写入和模型限流
摘要失败	SemanticProcessor re-enqueue 或 document status failed	区分临时模型错误和永久错误，必要时调低并发或修正文档格式
检索质量	召回过宽、答案引用不准、score 过低	检查 L0/L1 摘要质量、rerank 配置、 target_uri 和 context_type
记忆污染	重复记忆、过时偏好、不准确案例	启用去重合并，保留归档审计，定期做热度和低质记忆治理
权限边界	跨账号或跨用户检索异常	检查 RequestContext、owner_space、 account_id 和 HTTP 身份头
增量同步	旧内容未删除或新内容未索引	检查 target_uri、生命周期锁、watch task 和 diff 同步日志

实施建议

先以客服机器人或投标资料库作为高价值场景验证闭环，再把同一知识底座扩展到更多 AI 应用。每扩展一个场景，优先复用资源入库、记忆沉淀、检索增强和权限治理能力，只新增场景 Prompt、工具链和业务流程编排。

8. 源码依据与模块地图

模块	关键文件	说明
总体架构文档	docs/zh/concepts/01-architecture.md	服务层、处理层、双层存储和数据流说明
上下文层级	docs/zh/concepts/03-context-layers.md	L0/L1/L2 分层设计
资料抽取	docs/zh/concepts/06-extraction.md	Parser、TreeBuilder、SemanticQueue 和多格式支持
检索设计	docs/zh/concepts/07-retrieval.md	find/search、TypedQuery、分层检索和重排
会话与记忆	docs/zh/concepts/08-session.md	Session commit、归档、记忆抽取和去重
多租户	docs/zh/concepts/11-multi-tenant.md	account/user/agent 隔离和身份边界
资源 API	openviking/server/routers/resources.py	add_resource、temp_upload、knowledge documents
搜索 API	openviking/server/routers/search.py	find、search、grep、glob 路由
文档树落位	openviking/parse/tree_builder.py	临时目录、URI 解析、唯一命名
语义处理	openviking/storage/queuefs/ semantic_processor.py	L0/L1 生成、增量更新、熔断重试、向量化
会话实现	openviking/session/session.py	消息、归档、异步提交和 active_count

记忆抽取	openviking/session/memory_extractor.py	8 类记忆候选和工具调用格式化
记忆压缩	openviking/session/compressor.py	去重合并、写入、切块与语义刷新
客户端	openviking_cli/client/*.py	Local/HTTP/Sync Client 封装

以上模块共同构成 OpenViking 的通用知识库能力。对参赛展示而言，可把客服机器人作为“已验证落地案例”，把智能投标作为“可复制扩展案例”，核心亮点则落在统一知识底座、长期记忆、分层检索和多应用复用。